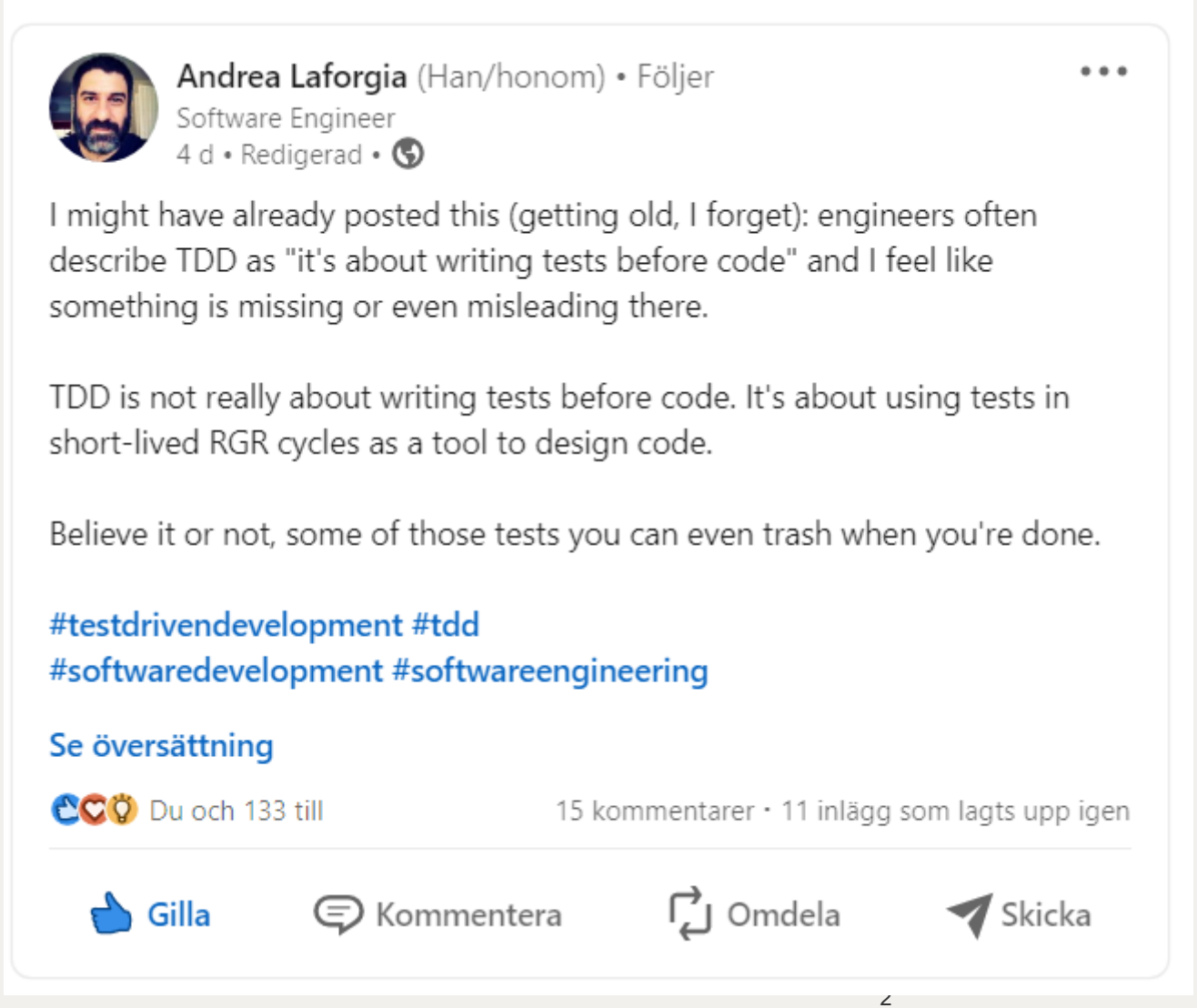
knowit

Test driven development

By example

TDD myths

knowit



Andrea Laforgia (Han/honom) • Följer
Software Engineer
4 d • Redigerad •

I might have already posted this (getting old, I forget): engineers often describe TDD as "it's about writing tests before code" and I feel like something is missing or even misleading there.

TDD is not really about writing tests before code. It's about using tests in short-lived RGR cycles as a tool to design code.

Believe it or not, some of those tests you can even trash when you're done.

#testdrivendevelopment #tdd
#softwaredevelopment #softwareengineering

Se översättning

Du och 133 till — 15 kommentarer · 11 inlägg som lagts upp igen

Gilla | Kommentera | Omdela | Skicka

TDD myths

knowit



Jason Gorman • 2:a
Of @codemanship. Trains and mentors software developers, and hel...
4 d • Redigerad •

+ Följ ···

One criticism of "strict TDD" is that you're not allowed to add tests after the code's complete. I don't recognise this version of "strict" TDD.

If the tests I'm left with from TDD-ing code aren't giving me that warm, fuzzy feeling, I'll explore more, maybe write more tests (or refactor to data-driven or property-based tests if it really has to work, or grab a colleague and do guided inspections if it really, really has to work, or even a proof of correctness if it really, really, really has to work. Whatever gives me sufficient warm fuzzies, basically).

As far as I'm aware, after ~25 years of doing and teaching TDD, that's allowed. Not just allowed, in fact, but recommended. 5/5 on Tripadvisor. Would go again.

And likewise, if I think, going forward, I don't need some of these regression tests, I'll remove them or consolidate etc. The tests that drive our design often correspond to good regression tests, but certainly not always. More work usually needs to be done. Once the design's in place from TDD-ing the code, we're in the realms of something called "testing", where tests we add are expected to pass first time (though they might not, and that's a good reason to do it).

Oh, and this is probably a good time to mention mutation testing. I'll routinely mutation test new or changed code and if that throws up gaps in my regression tests, I'll fill them.

TDD myths



Tim Ottinger • 2:a
Here (on LI and in Scotland) to help if I can.
2 v •

+ Följ

"TDD is writing the tests first" is a terrible phrase. We need to do something about that.

You don't write "the tests" first. That is a plural, and often when it is expressed that way people imagine the same tests that they would have written after the code was completed. That's all wrong.

You write one test, then implement enough code to pass that test, then refactor the existing code and test(s) if necessary, and then maybe commit or integrate before you get too much divergence from the main branch.

You don't write "the tests" first. You write a test first.

The test you write isn't a test of the code you haven't written. It is a test of what you want the code to do (that it doesn't do yet). It's a whole different mindset over writing tests to get coverage for existing code.

I lack the imagination and genius to write all of the test-last tests before writing the code. I have no idea how anyone would do that.

But I have ample skill and knowledge to write a test for what I want to happen before I write the code that makes it happen.

Do I want to throw an exception on bad input? Cool - here's a specific bad input, and here's the expected exception. We can do this for a lot of kinds of bad inputs, but we'll take it one kind of badness at a time.

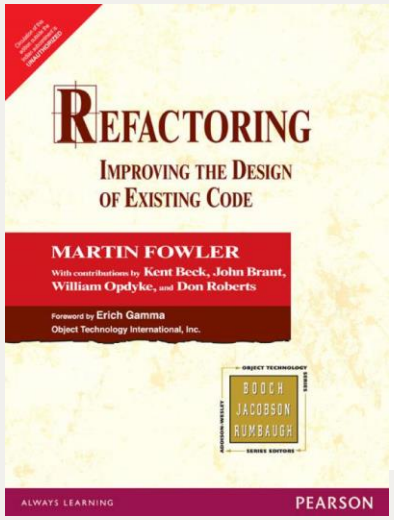
knowit

knowit

WHOAMI

- Sebastian
  - / Developer
  - / Trainer
  - / Manager

“I’m not a great programmer; I’m just a good programmer with great habits.”





Definition

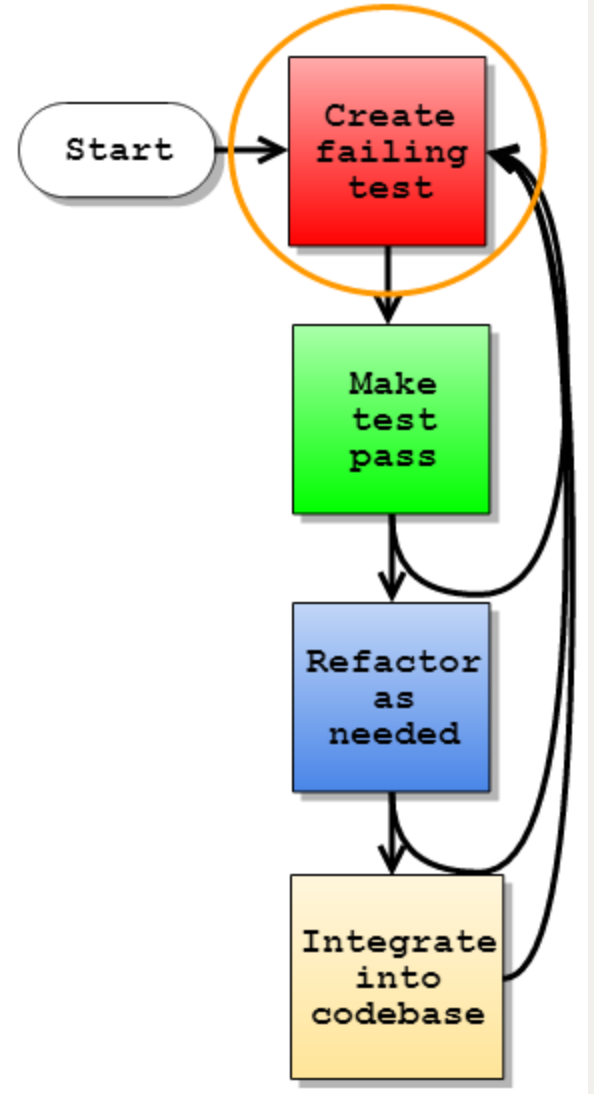
knowit

- Development technique
- Breaking down a problem into pieces
- Rapid feedback
- Design process

- Design results
  - Dependencies are injected
  - Decoupled components
  - Modular and extensible system
  - Frequent usage of composed method

- Consumption awareness
- Verify that the test is useful
- Improves testability

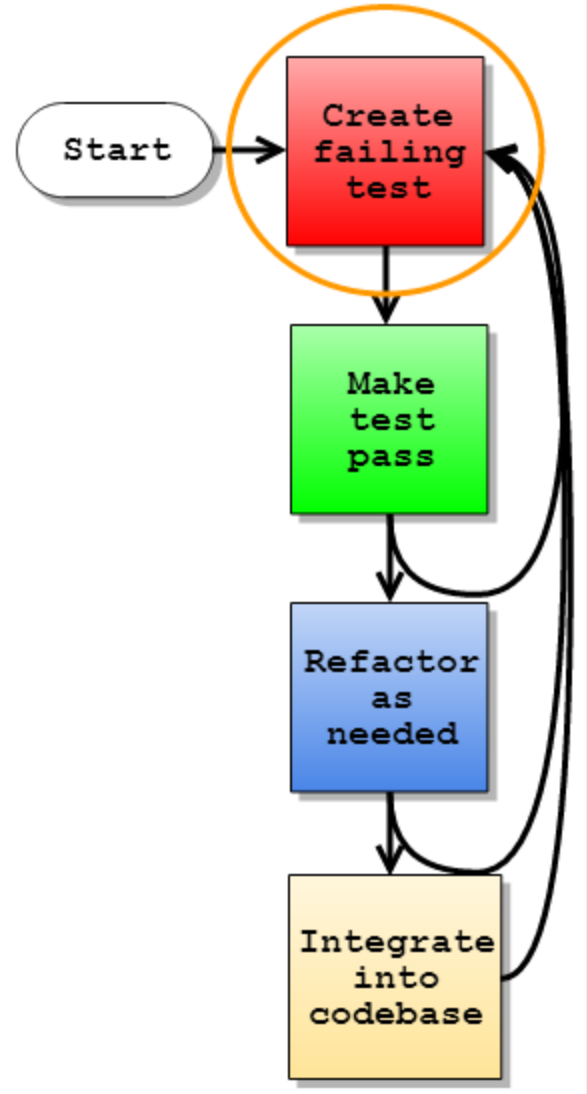
```
@Test
public void simpleAddition() {
    Calculator calculator = new Calculator();
    int expected = 3;
    calculator.add(1,2);
    assertThat(expected, is(expected));
}
```



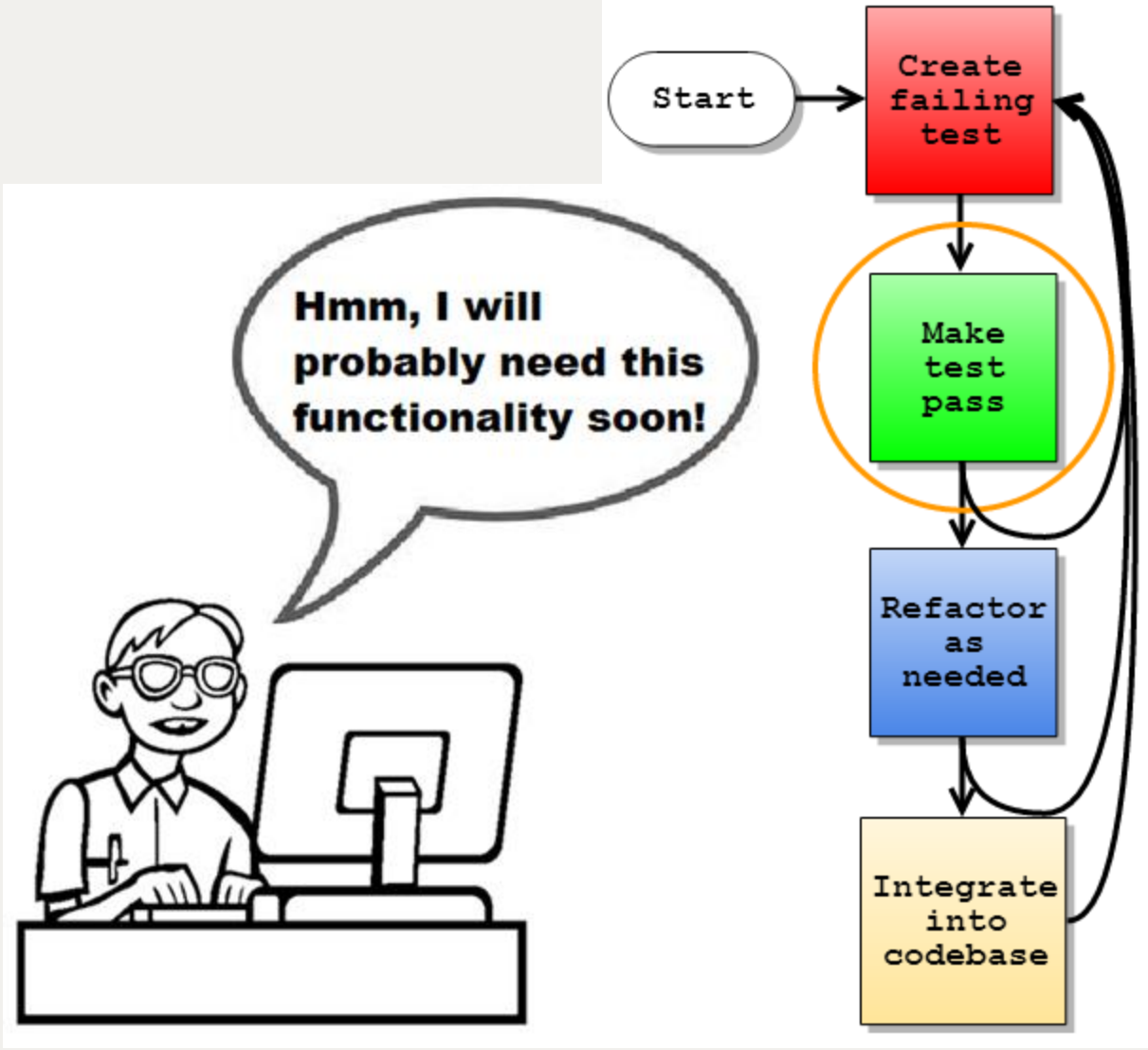
knowit



```
@Test
public void detects_adjacent_mine() {
    Mineplacer mineplacer = new Mineplacer() {
        public String contructMineField() {
            return      "[*,*,*,*]"
                    +   "[*,X,*,*]"
                    +   "[*,*,*,*]"
                    +   "[*,*,*,*]";
        }};
    MineField mineField = new MineField(mineplacer);
    assertEquals(1, mineField.atPos(0, 0));
}
```



- Implement the change
- Means done, although maybe not
- Prevents gold plating



```java
interface MessageStrategy {
    public void sendMessage();
}
abstract class AbstractStrategyFactory {
    public abstract MessageStrategy createStrategy(MessageBody mb);
}
class MessageBody {
    Object payload;
    public Object getPayload() {
        return payload;
    }
    public void configure(Object obj) {
        payload = obj;
    }
    public void send(MessageStrategy ms) {
        ms.sendMessage();
    }
}
public class MyProgram {
    public static void main(String[] args) {
        MessageBody mb = new MessageBody();
        mb.configure("Hello World!");
        AbstractStrategyFactory asf = DefaultFactory.getInstance();
        MessageStrategy strategy = asf.createStrategy(mb);
        mb.send(strategy);
    }
}
class DefaultFactory extends AbstractStrategyFactory {
    private DefaultFactory() { }
    static DefaultFactory instance;

    public static AbstractStrategyFactory getInstance() {
        if (instance == null)
            instance = new DefaultFactory();
        return instance;
    }
    public MessageStrategy createStrategy(final MessageBody mb) {
        return new MessageStrategy() {
            MessageBody body = mb;
            public void sendMessage() {
                Object obj = body.getPayload();
                System.out.println((String) obj);
            }
        };
    }
}
```

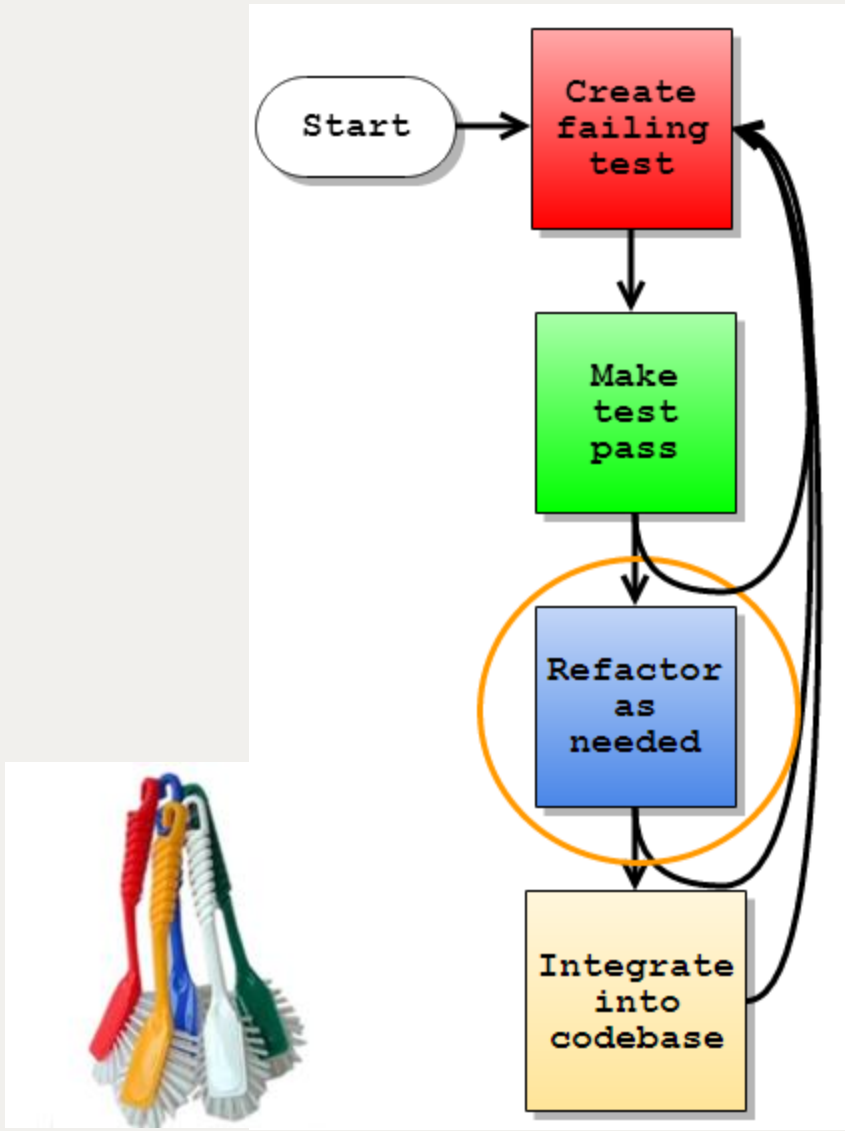
knowit

- Refactoring != redesign
- As simple implementation as possible for todays needs
- Easier when backed by tests

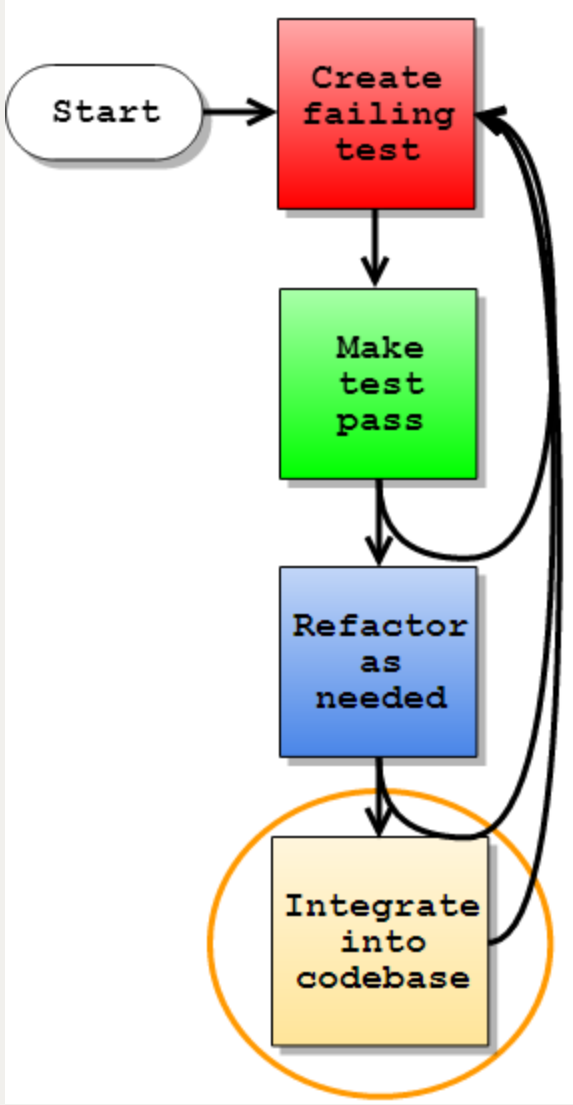
```
    //...
    Person p = getPerson(socialSecNr);
    //print the person
    System.out.println(p.getFirstName());
    System.out.println(p.getLastName());
    System.out.println(p.getSocialSecNr());
}
```



```
    //...
    Person person = getPerson(socialSecNr);
    print(person);
}
```



Merge into source control



knowit

Brain dumping ground



knowit

Merciless refactoring



The todo list

knowit

- Dynamic buffer
- Keeps noise away
- Examples of test cases
- Test order has a huge impact on TDD flow.

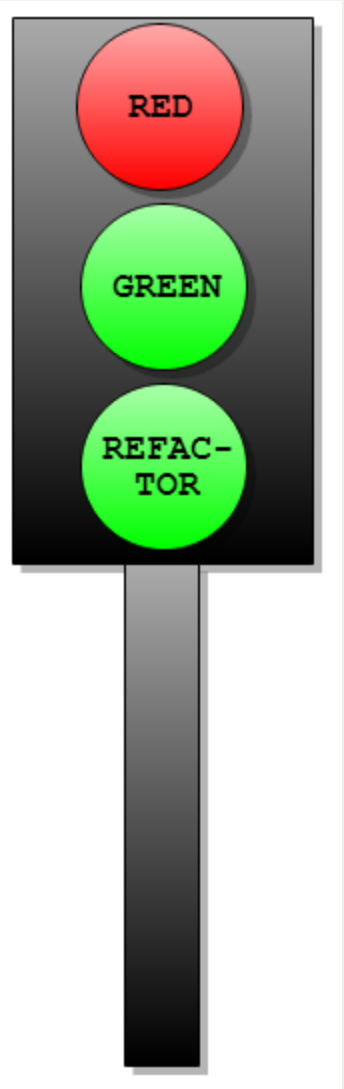


TDD demonstration

knowit

- “The cards game”

https://www.codewars.com/kata/53417de006654f4171000587



knowit